CRAFT: A Structural Conformance Index for System Resilience, and the Evaluator That Did Not Exist

Rudson Kiyoshi Souza Carvalho Independent Researcher ORCID: 0009-0009-1369-7517

Preprint v0.9 — empirical results consolidated; agent-auditor agreement study and full sensitivity analysis identified as future work in §9.

Abstract

Resilience assessment in distributed systems is dominated by outcome metrics — Mean Time Between Failures (MTBF), Service Level Indicators (SLI), load and chaos experiments. These share a structural dependency: they can only observe what the operating environment happened to exercise. A system that was never stressed and a system engineered to withstand stress produce indistinguishable readings. This paper presents CRAFT (Conformance-based Resilience Assessment of Fault Tolerance), a method that scores resilience as *structural conformance* — the presence and calibration of protection mechanisms at a system's functional boundary — rather than as observed outcome. CRAFT assigns weighted scores to protection mechanisms across five evaluation verticals, admits negative weights for configurations that worsen failure behaviour, applies a coherence degradation factor penalising multi-domain services, and normalises the result to a 0–10 index (CRAFT Score — CRAFT Score) directly interpretable as a service level tier. The weight tables are not fixed by the model: they constitute a reference profile, and the paper specifies an incident-anchored protocol by which an organisation derives its own.

The framework was specified in 2023 and shelved. Its original specification documents the reason: obtaining a precise index required an evaluator capable of reading source code semantically — mapping processing flow, resolving framework autoconfiguration and dependency injection, and understanding object interaction — and the specification states plainly that such a tool did not exist on the market, recommending manual counting instead. Manual counting made the framework unusable at organisational scale.

This paper's contribution is not the model alone but the resolution of that documented obstacle. Code-comprehending agents now satisfy the 2023 requirement, and they do so with three properties manual counting never had: they read configured *values* rather than mechanism presence, which closes the checkbox-conformance gap and makes negative and conditional weights computable for the first time; they place the evaluator outside the evaluated team, removing the incentive that corrupts self-assessment; and they evaluate continuously, converting the index from a periodic photograph into a per-commit time series in which resilience regression is attributable to the change that caused it. We report nine service evaluations across five public repositories (mean CRAFT Score 2.00, range 0.00-10.00), identify four structurally distinct failure shapes among the low-scoring services, and document one arithmetic defect in the measurement profile itself, caught by the positive-control fixture and corrected before results were finalized. We further show that CRAFT separates into a domain-specific evidence-collection layer and a domain-agnostic scoring layer, making the model portable to regulated domains where low traffic volume renders outcome metrics statistically vacuous, and we state explicitly the class of defects CRAFT does not detect.

Keywords: resilience engineering, structural conformance, design assurance, microservices, automated code audit, AI agents, production readiness, software governance

1. Introduction

Ask an engineering manager whether a system is reliable and the answer is a list. It has timeouts. It has health checks. It uses the platform team's error-handling library. It has a rate limiter. The list is assembled from memory, is not comparable to any other team's list, and terminates in a non-answer for anyone outside the engineering function: *probably fine, then.*

This is not a communication problem. It is a measurement problem wearing a communication problem's clothes. There is no scalar to report, so a list is reported instead.

The industry does have scalars. MTBF reports how long a system ran between failures. SLIs and SLOs report the fraction of requests served within objective. Chaos experiments report survival under injected turbulence. Load tests report behaviour under simulated demand. All four are valuable and all four share a limitation that is rarely stated as such: they are conditioned on what the environment, or the experimenter, happened to exercise.

A system operating in a stable environment accumulates an excellent MTBF regardless of how it was built. A service with a 300-second timeout on a downstream dependency will show a pristine SLO until the day that dependency

degrades slowly rather than failing fast. A chaos experiment reveals the failure modes the experimenter thought to inject; the mode nobody imagined remains unexamined and unreported. MTBF, in this framing, is a metric of luck as a function of time. It cannot distinguish a genuinely well-engineered system from one that was fortunate in its operating conditions during the observation window.

Other engineering disciplines resolved this asymmetry decades ago, and resolved it by refusing to wait. Aviation software certifies against DO-178C, automotive functional safety against ISO 26262, and medical devices against IEC 62304 — regimes that assess *how the artefact was constructed* precisely because assessing it by outcome would require the accident to happen first. This is the distinction between **design assurance** and **operational evidence**. It is well established outside software engineering and largely absent within it, where "reliability measurement" is assumed to mean measuring production.

CRAFT (Conformance-based Resilience Assessment of Fault Tolerance) is a design assurance instrument for distributed systems. It does not observe the system running. It reads how the system was built, scores the protection mechanisms present at its functional boundary, penalises architectural incoherence and known anti-patterns, and normalises to a 0–10 index — the the CRAFT Score. The index answers, at day zero and before any traffic exists, the question the list of mechanisms was failing to answer.

CRAFT was specified in 2023 in an industrial banking context and was not adopted. The reason is documented in the original specification and is the pivot of this paper. Section 4 examines it in detail; briefly, the specification identifies two attainable indices. A *macro* index could be automated with static analysis tools of the period — counting the presence of annotations, starters, and configuration properties — but the specification explicitly downgrades this to an indicator, because presence of an annotation is not evidence of correct implementation. A *precise* index would require an evaluator capable of interpreting source code and its dependencies, mapping the processing flow sequence, understanding framework autoconfiguration and dependency injection, recognising design patterns in use, and modelling how objects interact. The specification concludes that such a tool did not exist on the market and recommends manual counting by an analyst or auditor.

That recommendation was the framework's ceiling. Manual expert counting is defensible for one service and impossible for four hundred.

The requirement stated in 2023 is a description of a code-comprehending agent. It was written before such systems were practically available. It is now satisfiable. This paper therefore makes four claims:

1. Resilience admits a structural conformance measure that is independent of operating conditions, and CRAFT is a concrete instantiation of one (§3).
2. The framework's non-adoption was a cost-of-evidence failure, not a model failure, and this is documented rather than reconstructed (§4).
3. Agent-based auditing resolves the obstacle along three axes manual counting could not reach — semantic depth, evaluator independence, and continuity — and semantic depth in particular enables scoring constructs (calibration bands, negative weights, conditional weights) that the model anticipated but could not previously compute (§5).
4. The model separates cleanly into evidence collection and scoring, making it portable to regulated domains where low traffic volume renders outcome-based metrics statistically vacuous (§7).

We also state what CRAFT does not measure, at length and by name (§8). A conformance index presented as complete invites exactly the misuse that discredits conformance indices.

---

2. Background and Related Work

2.1 Outcome-based reliability metrics

MTBF and MTTR (Mean Time To Repair) originate in hardware reliability engineering, where component failure is stochastic and population statistics are meaningful. Their transfer to software has been criticised on the grounds that software failure is deterministic given input and state; the randomness lies in the environment, not the artefact. Littlewood and Strigini's classical analysis of the limits of software reliability quantification [1] articulates why environment-conditioned statistics cannot certify rare-failure regimes; software reliability growth modelling in the Musa tradition [2] remains valuable where failure data is plentiful, which is precisely the regime this paper's target domains lack.

2.2 Service level objectives and Site Reliability Engineering

The SRE literature formalises reliability as an error budget against an SLO, with SLIs measured from production

traffic. This is the dominant industrial frame. [3] Its principal assumption is traffic volume sufficient for the SLI to be statistically meaningful — an assumption that fails for low-volume, high-consequence services, a case Section 7 develops.

The SRE canon does include a structural instrument: the Production Readiness Review (PRR). The PRR is a qualitative checklist applied as a launch gate. CRAFT is best positioned as a *quantified, ordinal, and composable PRR* — comparable across teams, aggregable across a journey, and re-evaluable continuously rather than once at launch. This positioning matters: CRAFT is not a competitor to SLOs and should not be presented as one.

2.3 Chaos engineering

Chaos engineering builds confidence in a system's capacity to withstand turbulent production conditions through controlled experimentation. [4] CRAFT and chaos engineering are complementary and were framed as such in the original specification: chaos injects disturbance to discover whether the system survives; CRAFT verifies that the mechanisms intended to survive disturbance were in fact built and calibrated. Chaos tests the hypotheses the experimenter formulates. CRAFT is indifferent to whether anyone formulated a hypothesis.

2.4 Design assurance in regulated engineering

Aviation software certifies against DO-178C [5], automotive functional safety against ISO 26262 [6], and medical-device software against IEC 62304 [7]; ISO/IEC 25010 [8] provides the general product-quality vocabulary. Software engineering thus maintains a mature process-assurance tradition in safety-critical domains and an outcome-measurement tradition in web-scale services, and the two barely speak. CRAFT sits deliberately in the gap.

2.5 Architectural conformance checking

Static architecture validation tools — ArchUnit and comparable rule engines — verify that code obeys declared structural rules. The original CRAFT specification names ArchUnit explicitly as a candidate for the macro index and equally explicitly identifies its ceiling: rule engines confirm that a construct exists, not that its configuration is sound. [9]

2.6 Positioning

CRAFT's contribution against this background is a scoring model that is (a) computed from artefact structure rather than execution, (b) normalised to a bounded ordinal scale with declared service tiers, (c) composable across a multi-service journey, (d) sensitive to domain coherence via an explicit degradation factor and to configuration anti-patterns via negative weights, and (e) actionable — it emits a ranked remediation plan with projected index gain per action, which no outcome metric does.

---

3. The CRAFT Model

The model has two layers, and keeping them distinct is what the rest of the paper depends on. The **scoring layer** — verticals, weighted summation, boundary-relative normalisation, degradation, journey composition, tiering — is the framework's contribution and is domain-agnostic. The **weight tables** are not part of the model: they are a *profile*, an instantiation of the model for a particular technology and threat environment, and they are expected to differ across organisations. This section presents the scoring layer (§3.1, §3.3–§3.8) and the reference profile with its derivation and replacement protocol (§3.2).

3.1 Evaluation verticals

A vertical is a group of protection mechanisms applicable to a class of interaction point. Five are defined in the base specification:

Code	Vertical	Definition
EE	External Entry	Interaction point with a client
SE	External Exit	Data sent to another system with functional dependency
CE	External Consultation	Integration with another system on which there is functional dependency
DI	Internal Data	Database, cache, or file access within the service's own domain that affects functionality
AC	Application Container	Runtime lifecycle and integrity configuration of the container

The set is extensible. The original specification anticipates domain-specific verticals — SE-KAFKA for producer-type asynchronous outputs is developed in Appendix A — and states that the model may be adapted to the idiomatic protection practices of other technology stacks.

3.2 Weights: Reference Profile MS-1 and the calibration protocol

3.2.1 Status of the tables

The weights presented in this paper constitute **Reference Profile MS-1** (microservices, profile 1). MS-1 derives from expert judgement over production incident experience in a tier-1 banking microservice estate, in which a standard set of priority protections had been established institutionally. The profile is an instantiation of the model, not a component of it. Organisations operating under different execution models, threat profiles, or platform primitives are expected to derive their own; §3.2.4 specifies a protocol for doing so, and §3.2.5 specifies the robustness check that any profile — including this one — owes its users.

Two consequences of this framing are worth stating plainly. First, disagreement with a specific weight in MS-1 is not disagreement with CRAFT; the model is indifferent to which defensible profile it runs on. Second, the framing imposes an obligation the original specification did not discharge: if profiles are replaceable, the framework must say how a replacement is produced. Sections 3.2.3–3.2.4 exist to discharge it.

3.2.2 Form of a profile

Each vertical carries a table of applicable protections and integer weights. The EE vertical of MS-1 illustrates the form:

Weight	Protection
0	None
3	Rate limiter *(one type only)*
5	Bulkhead *(one type only)*

The mutual-exclusion annotation is load-bearing: EE's maximum is 5, not 8, because a rate limiter and a bulkhead address the same concern by different means and applying both does not compound protection. Profiles may express two further relations beyond exclusion — *conditionality* and *negativity* — introduced in §3.3, because they change the model's arithmetic domain rather than merely its parameter values.

3.2.3 Semantic anchor: what a weight level means

A profile's weights are summed, and summation presumes a cardinal scale: a weight of 4 must mean *twice the contribution* of a weight of 2, not merely *a higher priority than*. Weights elicited as priorities are ordinal, and summing ranks is not a valid operation. MS-1's weights were elicited from production priority judgements; to license their summation, the profile binds each level to a declared contribution class:

Weight	Contribution class	Criterion
5	**Function preservation**	Keeps the functionality operating when the dependency fails entirely
4	**Resource containment**	Prevents the failure from exhausting the service's shared resources
3	**Blast-radius limitation**	Reduces propagation of the failure beyond its point of origin
2	**Recovery acceleration**	Shortens the return to healthy state
1	**Marginal**	Real but small gain, or gain conditional on other protections
0	**Null**	No resilience effect
< 0	**Anti-pattern**	Worsens failure behaviour relative to absence (§3.3)

The anchor makes each MS-1 assignment derivable rather than stipulated: fallback preserves function under total dependency failure (5); timeout prevents pool and thread exhaustion (4); a circuit breaker limits blast radius (3); a connection pool accelerates recovery through isolation and reuse (2); replication of internal data preserves function (5); a bulkhead outranks a rate limiter at EE (5 vs 3) because partition isolation preserves function for unaffected consumers, whereas rate limiting only bounds the radius. The anchor also caps each mechanism: no calibration exercise can assign a mechanism a weight above the ceiling of its contribution class.

The anchor was constructed after the MS-1 weights existed and was found consistent with them — stated here plainly because it is evidence that the original production judgements encoded a coherent implicit criterion, not a post-hoc rationalisation of arbitrary numbers.

3.2.4 Incident-anchored calibration protocol

An organisation derives its own profile as follows.

1. **Window.** Select 12–24 months of production incidents with an identified technical root cause.
2. **Attribution.** For each incident, determine whether the presence of a mechanism in the candidate table would have *prevented*, *contained*, or merely *shortened* the event. Incidents with no attributable mechanism are set aside and counted: the set-aside rate measures the table's coverage, and a high rate indicates missing verticals rather than a calibrated table.
3. **Raw scoring.** For each mechanism m: $\text{score}(\ m) \ = \sum_i \text{severity}_i \times \text{effect}_i$ over incidents where m was the absent mechanism, with severity on the organisation's existing incident scale and effect class fixed at prevented = 1.0, contained = 0.6, shortened = 0.3.
4. **Normalisation.** Map raw scores to the 0–5 scale by rank *within each contribution class of the anchor* — the anchor sets each mechanism's ceiling; the incident data orders mechanisms beneath their ceilings.
5. **Review.** An expert panel may adjust any weight by ±1 with recorded justification. Mechanisms with no incident history receive a weight by analogy and are flagged *uncalibrated*.
6. **Re-evaluation.** Recalibrate annually or upon platform change.

The protocol converts CRAFT from a framework that ships a table into a framework that manufactures tables from the organisation's own failure history — which is also the only defensible source of cardinal weights available to most organisations.

Disclosure. MS-1 itself was *not* derived by this protocol; the protocol postdates it. MS-1 is calibrated expert judgement, and the protocol is proposed, not validated.

3.2.5 Sensitivity: what configurability obligates

If profiles are replaceable, the framework's conclusions must be shown to survive reasonable disagreement between profiles. The check is: perturb each MS-1 weight by ±1 and ±2 across the evaluated corpus and report the fraction of services whose SLA tier assignment changes. A weight-perturbation analysis was run on the present corpus (±1 and ±2 on every weight) and returned zero tier changes across all nine services. **This result is reported but should not be read as evidence of robustness.** The corpus is polarised: eight services score between 0.00 and 2.97 — deep inside the Unsatisfactory band — and one scores exactly 10.00. Only one service (2.97) sits near a tier boundary at all. A perturbation test has discriminating power only when scores sit near thresholds, and this corpus provides almost no such cases; the null result is an artifact of the score distribution, not a property of the model. A meaningful sensitivity analysis requires a corpus with services distributed across the Acceptable and Good bands, which §9

records as an open sampling problem. The same limitation applies to the degradation-base curve: every service in this corpus has D=1, so the factor is inactive throughout and 0.85/0.90/0.95 are indistinguishable by construction. The same treatment applies to the degradation base (§3.5).

3.3 Negative and conditional weights

Two scoring constructs extend the profile form beyond positive summation. Both were latent in the original specification — which noted that the tables were a starting point refinable by configuration bands such as *timeout up to 1s (weight 3), up to 2s (weight 2)* — and both were uncomputable under the evidence available at the time, for reasons Section 5.1 makes precise.

Negative weights. The additive model implicitly assumes that the worst configuration is absence. It is not. Several configurations degrade failure behaviour relative to having nothing:

- *Timeout inversion* — a call timeout longer than the caller's own: the caller abandons first while the resource remains held, consuming pool capacity for no benefit (−3);
- *Unconditional liveness* — a probe returning success regardless of state: it disables the orchestrator's restart path, which is strictly worse than having no probe (−3);
- *Circular fallback* — a fallback that consults the same degraded dependency or one sharing its backend, doubling load on the failing system (−2);
- *Inert circuit breaker* — thresholds or windows configured such that the breaker cannot trip in practice, providing auditable assurance of a protection that does not exist (−2);
- *Retry without backoff* — the direct mechanism of retry storms; see §3.3.1 (−2);
- *Pool without timeout* — the pool then merely determines how long exhaustion takes (−1).

With negative weights, $\mathrm{Index}_{\min}$ is no longer zero: it is the sum of the penalties applicable to the topology under evaluation. The normalisation of §3.6 requires no modification — $\mathrm{Index}_{\min}$ was always a parameter of the topology, never a constant. The model was already general enough; this direction had simply never been exercised.

Conditional weights. Protections are summed as if independent, and some are not. A circuit breaker without a timeout is nearly inert: breakers trip on failure, and a slow-but-alive dependency produces waiting, not failure — without a timeout there is nothing to count and the breaker never opens. MS-1.1 therefore extends the exclusion relation of §3.2.2 with a prerequisite relation, written `mechanism w [req: prerequisite, else w′]`:

Protection	Prerequisite	Weight absent the prerequisite
Circuit breaker	Timeout	1 (instead of 3)
Fallback	Timeout or circuit breaker	2 (instead of 5)
Self-healing	Specific liveness probe	0 (instead of 3)
Retry	Timeout	0 (instead of its band value)

The arithmetic remains a weighted sum; only the weight is resolved in context.

3.3.1 A worked interaction: retry and idempotency

The profile's most instructive correction concerns retry. MS-1 originally assigned retry an unconditional weight of 1. Retry without exponential backoff and jitter is the direct cause of retry storms and a contributor to metastable failure — a mode Section 8 acknowledges structural analysis cannot fully see. An unconditional positive weight therefore *rewarded the mechanism that produces the failure mode*. MS-1.1 replaces it with a calibration band: no retry (0); retry without backoff or without an attempt cap (−2); retry with fixed backoff and a cap (1); retry with exponential backoff, jitter, and a cap (2); a global retry budget adds a further +1.

Retry interacts with write semantics: a retried non-idempotent write duplicates its effect. In MS-1 the combination of two individually scored items produced a defect the table could not see. MS-1.1 adds idempotent write (4) to the SE vertical — the property was previously scored only in the SE-KAFKA extension, though it is equally critical for synchronous writes — and penalises retry enabled over a non-idempotent operation (−2).

Both corrections require the evaluator to read configured values and trace write semantics. Neither is expressible as an annotation-presence rule. This is not incidental; it is Section 5's argument in miniature.

3.4 Boundary determination

Measurement is performed from the internal perspective of the application under evaluation, against its non-functional requirements. This requires an explicit **boundary**: which integrations and components count toward the index.

The specification's rule is that points are counted for integrations or components *at the frontier of the target application which, when they fail, compromise the application's functioning*. A dependency belonging to another system is excluded, on the stated grounds that the evaluated team has no authority to remediate it.

This is not a simplifying convenience. It is a governance commitment embedded in the metric: **CRAFT scores only what the evaluated party has authority to change.** An index that penalised a team for defects outside its remit would be a report of organisational structure, not of engineering quality, and would be correctly ignored. The boundary rule is what makes the index actionable, and it makes CRAFT a per-service measure that composes upward (§3.7) rather than a system-wide aggregate that dissolves accountability.

3.5 Raw index and degradation factor

For verticals $V_1 \dots V_n$ with resolved weights $W_1 \dots W_n$ (resolution per §3.3):

$$\text{Index} = \sum_{i=1}^{n} (\ V_i \times W_i)$$

A service that serves multiple functional domains — a *micromonolith* — shares its resources across those domains. Latency in one function can exhaust the resource pool serving another. Protection mechanisms present in the service do not protect against this, because the contention is internal to the service. CRAFT penalises the condition multiplicatively, before normalisation. For D functional domains served by one service:

$$\text{Index}_{\text{degraded}} = \left[\sum_{i=1}^{n} (\ V_i \times W_i) \right] \times 0.9^{(\ D-1)}$$

At $D = 1$ the factor is unity. Each additional domain removes 10% of index quality.

On the base 0.9. The base is a **declared convention, chosen for interpretability**, not a calibrated constant: ten per cent per domain is a quantity an architecture board can debate, adjust, and own. The convention's consequences are severe by design — at $D = 19$ the factor is 0.150, an 85% reduction, and Section 6.2 presents a production service whose score falls from 4.90 to 0.74 under it. What the paper owes the reader is not a derivation of 0.9 but evidence about the convention's influence: a sensitivity curve for bases 0.85/0.90/0.95; this was attempted on the present corpus and is uninformative because every audited service has D=1, leaving the factor inactive (§3.2.5). A refinement in which the base varies with the degree of resource coupling between domains — process-only co-residence being less hazardous than a shared connection pool and cache — is deferred to future work (§9): it would add an uncalibrated parameter to a model that already carries the honest burden of one convention.

The degradation factor is, notably, the only component of the model that detects an *architectural* defect rather than a missing or miscalibrated mechanism, and the only one whose remediation is a decomposition rather than an addition. Section 6.2's case terminates in exactly that recommendation.

3.6 Normalisation

$$\text{CRAFT} = \left(\frac{\text{Index} - \text{Index}_{\text{min}}}{\text{Index}_{\text{max}} - \text{Index}_{\text{min}}} \right) \times 10$$

$\text{Index}_{\text{min}}$ and $\text{Index}_{\text{max}}$ are computed for the specific topology under evaluation — the minimum and maximum attainable given that service's actual set of interaction points, with $\text{Index}_{\text{min}}$ incorporating any applicable anti-pattern penalties (§3.3). The index is therefore *relative to what this service could have achieved*, which is what makes scores comparable across services of different shapes.

Procedure:

1. Compute $\text{Index}_{\text{min}}$ for the application's topology, including applicable penalties
2. Compute $\text{Index}_{\text{max}}$ for the application's topology
3. Compute Index from implemented protections at their resolved weights, applying the degradation factor where $D > 1$
4. Normalise

3.7 Journey index

A functional journey traverses multiple services. Individual IRCs compose through a normalised weighted mean. For n services with journey weights W_i (unity when services are equally critical):

$$\text{CRAFT}_{MP} = \left(\frac{\sum (\ \text{CRAFT}_i \times W_i) \ - \sum \left(\text{CRAFT}_{\text{min},i} \times W_i \right)}{\sum \left(\text{CRAFT}_{\text{max},i} \times W_i \right) \ - \sum \left(\text{CRAFT}_{\text{min},i} \times W_i \right)} \right) \times 10$$

Since each service's CRAFT Score is already normalised to [0, 10], $\mathrm{CRAFT}_{\min,i} = 0$ and $\mathrm{CRAFT}_{\max,i} = 10$.

Example. A journey `client → gateway → Service X → Service Y` where $\mathrm{CRAFT}_X = 7$, $\mathrm{CRAFT}_Y = 6$, weights unity:

$$\mathrm{CRAFT}_{MP} = \frac{13 - 0}{20 - 0} \times 10 = 6.5$$

The journey scores in the *Good* tier. The composition is deliberately not a minimum function: a journey is not only as strong as its weakest service, because compensating protection in an adjacent service is real protection. The weight vector is the mechanism for expressing that some services are load-bearing in ways others are not.

3.8 Service level scale

CRAFT Score	Tier	Interpretation
8.0 – 10.0	Excellent	High reliability and resilience; no additional-domain overhead
5.0 – 7.9	Good	Reliable, but contains single points of failure without fallback, or unprotected adjacent integrations
3.0 – 4.9	Acceptable	Room for improvement; corrective measures required; low robustness
< 3.0	Unsatisfactory	Requires revision; risks damage to adjacent services and to the system as a whole

The scale converts a technical measurement into a term usable in a governance conversation. The failure mode it addresses is not engineering ignorance but the absence of a shared unit: *"is the system reliable?"* answered as *"it has an CRAFT Score of 8"* is a different institutional act from the same question answered with a list of mechanisms.

---

4. The Evaluation Bottleneck

The preceding section describes a model that is arithmetically simple. Nothing in it requires unusual tooling. The framework nevertheless did not scale, and the reason is stated in the 2023 specification rather than reconstructed here.

The specification distinguishes two attainable indices.

The macro index. Automation is possible by counting the presence of mapped best practices — the presence of a `@Bulkhead` annotation, the use of an engineering-team starter, the existence of circuit breaker, fallback, and timeout configuration — using static code validation tools such as ArchUnit. The specification's own assessment of this index is a downgrade: it is *an index of macro value, an indicator*, tallied from the existence of annotations, properties, and dependencies, and it constitutes a succinct form of analysis. For precise analysis, an analyst or auditor must perform the process manually to guarantee a detailed determination of what the developer actually implemented.

The precise index. The specification then states what a precise automated index would require: a static analysis tool *capable of understanding the source code* and its dependencies; one that interprets and maps the processing flow sequence; that understands the application framework deeply, including autoconfiguration and dependency injection; that recognises the principal design patterns in use; and that models how each object interacts with the others. Its conclusion is unambiguous: this would be a substantial challenge, **it does not exist on the market**, only some useful tools that might assist the process — and therefore manual counting is recommended for a more precise index.

Two observations about this passage.

First, it is a **falsifiable requirement specification written before its satisfier existed**. It is not a retrospective claim that the framework was ahead of its time. It is a list of five capabilities, recorded contemporaneously, together with a market assessment and a fallback recommendation. The fallback — expert manual counting — is what made the framework organisationally unusable. Manual counting scales linearly in senior engineer hours against a service count that grows without bound. The framework was not rejected on its merits; it was priced out.

Second, the five capabilities are a description of a code-comprehending agent. Semantic source comprehension, flow mapping, framework-level reasoning about autoconfiguration and injection, pattern recognition, and inter-object interaction modelling are precisely the capability profile of contemporary agentic code analysis. The 2023 assessment was correct for 2023 and is no longer correct.

The framework's ceiling was never its model. It was the cost of evidence.

---

5. Agent-Based Conformance Auditing

The naive statement of this section's claim is that the audit is now automated and therefore cheap. That statement is true and is the least interesting part. Agent-based evaluation changes three properties of the measurement, and cost is the third.

5.1 Semantic depth: from presence to calibration

The most serious objection to any structural conformance index is checkbox conformance. If the index rewards the presence of a timeout, a team adds a timeout of 300 seconds, the index rises, and nothing has been protected. Presence is not calibration. This objection is correct against the macro index, and the original specification effectively concedes it — hence the recommendation of manual audit.

The original specification also anticipated the resolution without being able to implement it: it notes that the weight tables are a starting point, refinable with configuration bands — *timeout up to 1 second (weight 3), up to 2 seconds (weight 2), container memory limits up to 70%*. That refinement was infeasible under annotation counting, because an annotation's presence is visible to a rule engine while its effective value frequently is not — resolved through property files, profile overlays, environment variables, and framework defaults. It is entirely feasible for an agent that resolves configuration through the dependency graph. The agent does not observe that `@Timeout` is present; it resolves the effective value to `PT300S` and scores the band. It does not observe that a circuit breaker exists; it reads `failureRateThreshold` and `waitDurationInOpenState` and determines whether the breaker can trip at all.

Two consequences follow, and they are distinct.

First, CRAFT transitions from presence conformance to calibration conformance. What was specified in 2023 as a future refinement becomes the default mode of operation, and the checkbox objection loses its target: the checkbox is no longer what is counted.

Second, the scoring constructs of §3.3 become computable for the first time. Every negative weight is a claim about a configured *value or relation*, not about presence: timeout inversion is a comparison between two resolved durations across a call boundary; an inert circuit breaker is a judgement about threshold values against realistic traffic; a circular fallback is a reachability fact in the dependency graph; retry-without-backoff is a configuration field; retry-over-non-idempotent-write is a joint fact about a retry policy and the semantics of the operation it wraps. Conditional weights likewise require establishing the co-presence and co-calibration of two mechanisms. None of these are expressible as annotation-presence rules, which is why the additive, presence-based table was the best the 2023 evidence regime could support. The profile did not become more sophisticated because the model changed; it became more sophisticated because the evaluator did.

5.2 Evaluator independence

The second change is structural rather than technical, and it is the more important of the two.

Under manual counting, the evaluation is performed by the engineering team, or by an analyst embedded in it, and reported upward. The party scored and the party scoring are the same party or are institutionally adjacent. Every incentive in that arrangement points toward a generous reading of ambiguous evidence. This is not an accusation of bad faith; it is the ordinary behaviour of self-assessment under organisational pressure, and it is why financial audit is not performed by the finance department.

An agent evaluating a repository is outside that incentive structure. It has no performance review contingent on the result, no relationship with the reviewer, and no memory of the argument the team had last sprint about whether the fallback was really necessary.

This is the same architectural separation the author has argued elsewhere in the context of multi-agent governance — that connection infrastructure and governance authority are distinct layers, and that collapsing them destroys the governance property. Here the principle applies reflexively, to the framework's own measurement: **the scoring authority must be separable from the scored party, or the score is not evidence.** Agent-based evaluation does not merely reduce the cost of the audit. It is the first configuration in which the audit is structurally independent.

5.3 Continuity

Manual audit is periodic because it is expensive — quarterly at best, in practice on demand and rarely. A quarterly measurement is a photograph.

Per-commit evaluation is affordable and converts the index into a time series. This has one consequence worth

stating explicitly: **resilience regression becomes attributable to the change that caused it.** A pull request that removes a fallback, widens a timeout past its caller's, or introduces a second functional domain into a service produces an immediate, localised index movement. Under quarterly audit the same regression surfaces months later, attributed to nothing, and is typically discovered during an incident.

The degradation factor is particularly well suited to continuous evaluation, because domain creep is incremental by nature. No single pull request converts a coherent service into a micromonolith. Nineteen of them do, and no individual author experiences their own change as the one that did it.

5.4 Pipeline

The audit is implemented as a nine-step agent procedure, published with this paper (§6.8). Its inputs are a repository path and commit; its outputs are a per-service evidence ledger (JSON), a human-readable findings report, and a corpus-level summary. The procedure: (0) record repo/commit/stack; (1) inventory independently deployable services; (2) inventory interaction points per service and classify into verticals under the boundary rule, with excluded dependencies recorded and justified; (3) detect mechanisms and resolve their effective values through the full configuration chain (inline code, property files, profile overlays, environment defaults in manifests), preceded by a mandatory repo-wide indirect-protection sweep covering shared internal libraries, AOP/interceptor registration, and service-mesh manifests, whose negative result must be stated explicitly rather than assumed; (4-6) score, degrade, and normalise per §3; (7) emit findings with per-finding evidence, risk statement, a self-contained fix prompt executable by a coding agent, projected score per cumulative fix, and two mandatory disclosures — any gap between the plan's coverage and the full index, and any topology-imposed ceiling below 10.0; (8) write outputs, including a portfolio scorecard when more than one service is audited; (9) run honesty checks (every scored line has file-and-line evidence; unclassifiable points are counted, not dropped). The provenance-or-reject rule — no evidence, no score, item logged UNVERIFIED — is the same discipline as the author's provenance-gate work in specification-driven development, applied here to the audit itself.

5.5 Instrument calibration

An agent is an instrument, and an uncalibrated instrument produces numbers rather than measurements. Two requirements follow.

Agreement. The paper must report agent-versus-human-auditor agreement on a sample. The present corpus was audited by a single agent without a blinded human co-audit, and no agent-versus-human agreement figure is therefore reported; this is the paper's most important open validation item and is committed as the first follow-up study (§9.3). Until it exists, the per-item evidence ledger is the compensating control: every scored claim resolves to a file and line a human can check in minutes, which converts trust in the auditor into verification of the audit.

A disconfirming prior. The author's prior work on continuous architecture assurance reports a recurring empirical finding: automated decision gates consistently accepted structurally defective outputs, and only independent human audit detected the errors. That finding bears directly on this paper's central claim and cuts against it. It is cited here rather than omitted, and the pipeline addresses it through `[per-item evidence traceability / human audit of a sampled fraction / an explicit statement that the risk is unresolved and bounded by the sampling rate — state which, and honestly]`. A reviewer who finds this tension unaddressed will treat the omission as more damaging than the tension.

5.6 Goodhart exposure

If CRAFT Score becomes a team performance metric, gaming returns. It migrates from checkbox to code: mechanisms written to be recognised by the evaluator rather than to protect the system. Automating the auditor does not repeal Goodhart's law; it changes the address to which optimisation pressure is delivered.

The mitigation is deployment discipline rather than model design. CRAFT Score is diagnostic and belongs in engineering review, not in compensation or ranking. Section 9.1 records the removal, on these grounds, of an incentive programme present in the original specification.

---

6. Empirical Evaluation

6.1 Corpus and method

Nine services across five public repositories were audited by an agent executing the published audit procedure (Appendix C) under profile MS-1.1, corrected mid-corpus to MS-1.1.1 (§6.5). The corpus was chosen to span stacks and intent: two widely-used microservice reference architectures (Spring PetClinic Microservices, Java/Spring; Google Online Boutique, Go/C#/polyglot gRPC), one large academic benchmark used in chaos-engineering research

(train-ticket; one of its 47 services sampled), one single-pattern resilience tutorial (spring-circuitbreaker-demo, Resilience4j), and one purpose-built positive-control fixture (craft-golden-demo) engineered by the author to implement every mechanism in the profile against a real running stack (Redis Sentinel, Kafka, Avro, Kubernetes manifests, CI).

Every scored item carries file-and-line evidence with a resolved configuration value; items whose effective value could not be resolved in-repo were scored zero and flagged `UNVERIFIED` rather than credited from assumed framework defaults. Before scoring any service, each repository received a mandatory indirect-protection sweep (shared internal libraries, AOP/interceptors, service-mesh manifests) so that absence claims are checked absences, not unexamined ones.

6.2 Results

Service	Repository	Stack	CRAFT Score	Tier
customers-service	spring-petclinic	Java/Spring	0.00	Unsatisfactory
visits-service	spring-petclinic	Java/Spring	0.00	Unsatisfactory
vets-service	spring-petclinic	Java/Spring	0.00	Unsatisfactory
ts-preserve-service	train-ticket	Java/Spring+AMQP	0.05	Unsatisfactory
api-gateway	spring-petclinic	Spring Cloud Gateway	1.13	Unsatisfactory
checkoutservice	microservices-demo	Go/gRPC	1.72	Unsatisfactory
circuitbreaker-demo	spring-circuitbreaker-demo	Java/Resilience4j	2.17	Unsatisfactory
cartservice	microservices-demo	C#/gRPC+Redis	2.97	Unsatisfactory
craft-golden-demo	craft-golden-demo	Java 21 full stack	10.00	Excellent

Mean CRAFT Score 2.00; range 0.00–10.00. One penalty triggered corpus-wide (checkoutservice, unconditional liveness, −3); twelve items UNVERIFIED across four services; six items set aside as outside the profile's vocabulary.

6.3 Four structurally distinct failure shapes

The eight Unsatisfactory results are not one phenomenon. The evidence ledgers separate them into four root-cause shapes, which matters because a model that produced a single undifferentiated "bad" would be measuring less than it claims:

Pure absence (customers/visits/vets-service; ts-preserve-service). No protection mechanisms implemented at any interaction point. The train-ticket case adds a scaling observation: its orchestrator makes eleven downstream calls through one shared, unconfigured HTTP client. High fan-out inflates $\mathrm{Index}_{\max}$ (more points requiring protection) while the implemented index stays at zero — the ratio, correctly, punishes unprotected breadth. This is a different risk shape from low cohesion: the service serves one domain ($D = 1$; no degradation applies), and the model discriminates between the two without special-casing.

Asymmetric coverage (api-gateway). The service protects its lower-consequence call (visit history: circuit breaker + graceful fallback) and leaves its higher-consequence call (owner lookup: no timeout, breaker, fallback, or retry) entirely bare. A reviewer skimming the file sees "there is a circuit breaker in here" and reasonably infers protection; the per-point ledger disagrees with line-level evidence. This is the presence-heuristic failure §5.1 predicts, occurring naturally in a flagship reference codebase.

Active anti-pattern (checkoutservice). The gRPC health handler wired to both the Kubernetes readiness and liveness probes unconditionally returns `SERVING`, regardless of the state of the six downstream connections the service holds. This is the corpus's only triggered penalty, and it is qualitatively worse than absence: the mechanism exists, is wired, and manufactures false confidence while disabling the orchestrator's recovery path. A repo-wide search additionally confirmed zero circuit breakers and zero retry mechanisms anywhere in this repository's business logic — a grep-verified fact about one of the most widely taught microservice architectures in the industry, and direct evidence for this paper's opening claim that structural resilience goes unmeasured even where reliability is the stated subject matter.

Protection without a boundary (circuitbreaker-demo). The most instructive case. A tutorial repository stacks all four core Resilience4j annotations — `@TimeLimiter`, `@Retry`, `@CircuitBreaker`, `@Bulkhead` — on one method, backed by a fully calibrated configuration (exponential backoff, 50% failure threshold, real fallback). Under a presence-counting index — precisely the "macro index" the 2023 specification downgraded — this service scores at the ceiling; it has the highest density of annotated resilience patterns in the corpus. CRAFT scores it 2.17, because the method those

annotations wrap performs no I/O: the “remote call” is a string format behind an artificial random delay. Per the boundary rule there is no external-consultation point at all, and the verticals that do not exist are excluded from the denominator rather than scored as failures ($\mathrm{Index}_{\max} = 23$ for this topology, against 113 for the six-dependency checkout service). The one real protection present — the bulkhead guarding the service’s own entry concurrency — receives full credit. The author of this repository was not gaming an evaluator; he was teaching a library honestly, and in doing so produced the cleanest natural specimen of protection density divorced from a protectable boundary available in the open-source ecosystem.

6.4 Positive control

The eight results above raise a fair objection: an instrument that only ever reports failure may be measuring its own pessimism. The ninth service exists to answer it. `craft-golden-demo` is a fixture engineered to implement every mechanism in the profile — six interaction points across all six verticals, against real infrastructure — and to document its own expected outcome. The audit was performed under the fixture’s own explicit instruction that its evidence map is documentation, not scoring evidence: every mechanism was located and read in source before the map’s claimed value was consulted. The result is 94/94 points, CRAFT Score 10.0, zero penalties, zero unverified items — the instrument reaches its ceiling exactly when the evidence earns it, and reaches exactly the ceiling the fixture was engineered to demonstrate.

Two methodological caveats attach, and both are stated rather than smoothed over. First, the fixture is the author’s own construction; it demonstrates that the instrument *can* award high scores on merited evidence, not that independently produced excellent services exist and are recognised — the corpus contains no independent service in the Good or Acceptable tiers, and discriminant validity in the middle of the scale remains unestablished (§9). Second, the fixture caught a real defect in the instrument: §6.5.

6.5 The fixture as instrument test: a profile bug found and fixed

The fixture’s documentation observed that the MS-1.1 profile’s declared vertical maxima (CE = 15, SE = 21) did not equal the literal sum of the profile’s own weight tables once the global-retry-budget modifier is included (17 and 23 respectively), and noted that either resolution — capping at the declared maximum, or summing literally and clamping — converges on the same fixture outcome. Independent recomputation during the audit confirmed the inconsistency: an arithmetic error in the profile, present since its drafting. The profile was corrected to MS-1.1.1 (declared maxima raised to match the tables; no weight changed) and the correction is recorded in the profile’s changelog. A positive-control fixture catching a defect in the measurement instrument — rather than merely confirming it — is the strongest form of service such a fixture can render, and it is why the corpus results are reported under the corrected profile.

6.6 A cross-cutting portfolio finding

One pattern repeats in five of the eight Unsatisfactory services and deserves statement at the portfolio level rather than per-service: **health-check machinery exists but is either not connected to anything that acts on it, or is connected and cannot fail.** Actuator endpoints present with no compose/Kubernetes probe referencing them (four services); a wired probe backed by an unconditional handler (one service). In no audited service was a health check simply absent — the industry has universally adopted the *artifact* of health checking while frequently omitting the *connection* that gives it consequence. This is a calibration-class finding, invisible to presence-counting by construction, and it generalises: the fix is nearly identical across all five affected services.

6.7 What the remediation output discloses about itself

Each audit emits an ordered remediation plan with projected CRAFT Score per cumulative fix. Two disclosure rules were added to the procedure during the corpus work, both after the plan’s silence was caught misleading a reader, and both are now mandatory: (a) a plan that does not cover every remaining point must state what it omits and what the full-coverage score would be, so a cumulative table cannot imply a completeness it lacks; and (b) the attainable ceiling must be checked against the deployment topology before 10.0 is promised — for the api-gateway, three container-vertical points assume a Kubernetes-class orchestrator and the service ships only with docker-compose, making its in-place ceiling 9.4, a fact the plan must state rather than bury. Severity language and score movement are additionally required to be reconciled explicitly wherever they could read as contradictory: a gap can be the most operationally dangerous item on a service while moving an aggregate index by fractions of a point, and the report must say both things in one breath or it invites the inference that one of them is wrong.

6.8 Reproducibility

The audit skill (procedure, profile MS-1.1.1, output schema), all nine evidence-ledger JSONs, per-service reports, the portfolio scorecard, and the corpus summary table are published at https://github.com/RudsonCarvalho/craft-audit

(Zenodo DOI pending release). The positive-control fixture, including its own CI-verified healthy-path and failure-path demonstrations, is published separately at https://github.com/RudsonCarvalho/resilience-golden-demo. Every score in this section can be re-derived by pointing the skill at the recorded commit of each repository.

---

7. Portability

The temptation in a portability section is to enumerate industries. Enumeration is weak evidence and reads as overclaiming. The claim CRAFT can actually support is structural.

CRAFT separates into two layers. The *evidence collection* layer is domain-specific: it determines what constitutes an interaction point, what protections are applicable, and how their presence and calibration are established from the artefact. The *scoring* layer — weighted summation over a profile, coherence degradation, boundary-relative normalisation, tiering, and journey composition — is domain-agnostic. It operates on the output of the collection layer and has no commitment to microservices, to any particular framework, or to software. The profile mechanism of §3.2 is what joins them: porting CRAFT is deriving a new profile through the calibration protocol and replacing the collection layer, while the model is retained unchanged.

This is the same layer separation the author has argued for elsewhere in the governance of multi-agent systems: the substrate that gathers and connects is not the authority that evaluates, and conflating them destroys the portability of both.

7.1 The conditions under which structural conformance is the right instrument

Rather than listing domains, the paper states the conditions and lets readers match their own:

1. **Failure consequence is high and failure frequency is low.** Outcome metrics require failures to learn from. Where failures are rare and expensive, waiting for the sample is not a strategy.
2. **Traffic volume is insufficient for statistically meaningful SLIs.** A service handling hundreds of requests a day cannot support a 99.9% objective in any meaningful sense; the confidence interval swamps the target.
3. **Assessment is required before operation.** Regulatory approval, launch gates, and procurement decisions all require a reliability judgement at a point where no operational evidence exists.
4. **An auditable, reproducible basis for the judgement is required.** Structural conformance produces a traceable ledger — this mechanism, at this location, with this configured value, scoring this weight. Outcome metrics produce a number whose derivation is the environment.

Where all four hold, structural conformance is not a proxy for outcome measurement. It is the appropriate instrument, and outcome measurement is the proxy.

7.2 Instantiation

A worked instantiation for a second domain is deliberately deferred to a follow-up paper rather than sketched thinly here. The strongest candidate is critical-infrastructure control software (grid operations), where all four conditions of §7.1 hold sharply and the low-volume/high-consequence profile makes SLO-style measurement structurally inapplicable; medical-device software under IEC 62304 is the defensible alternative.

---

8. What CRAFT Does Not Measure

A conformance index presented without its exclusions will be applied outside them. This section is therefore stated as strongly as the contributions.

CRAFT does not measure correctness. A service may implement every protection in the profile, score 10, and return wrong answers. Resilience conformance is orthogonal to functional correctness.

CRAFT does not measure performance. A well-protected service may be slow. Nothing in the model rewards latency.

CRAFT does not measure emergent and systemic failure — with one narrowed exception. Metastable failure, cascading saturation, and correlated failure across ostensibly independent dependencies are properties of the running system under load, and a per-service structural score cannot see them; load testing and chaos experimentation can, and here the objection that structural assessment is insufficient is simply correct. The exception is narrow and worth stating precisely: the *structural precondition* of one emergent mode — retry storms — is detectable and penalised under MS-1.1 (§3.3.1). CRAFT cannot observe a storm; it can observe, and score negatively, the uncapped, unbackedoff retry configuration from which storms are made. Detecting a mechanism’s

precondition is not detecting the mode, and the distinction is maintained here deliberately.

CRAFT does not fully measure calibration correctness, even under agent evaluation. §5.1 establishes that an agent reads configured values rather than mechanism presence. It does not establish that the agent knows the *right* value for a given dependency, which is a function of that dependency's latency distribution — operational knowledge, not structural. The agent can flag a 300-second timeout as implausible and a timeout inversion as wrong by construction. It cannot determine that 800ms is correct and 1200ms is not, absent production data.

CRAFT does not measure operational maturity. On-call quality, runbook accuracy, incident response, and deployment safety are unmeasured and consequential.

CRAFT does not measure what falls outside the boundary. By design (§3.4). A journey may fail entirely at a dependency correctly excluded from every constituent service's index.

The index is bounded above by its profile. A protection absent from the profile scores zero, and a novel or domain-idiomatic mechanism is invisible until the profile is extended. The set-aside rate of the calibration protocol (§3.2.4, step 2) is the model's own estimate of this blindness, and reporting it is part of using the framework honestly.

The correct composite posture is therefore: **CRAFT for design assurance, chaos and load experimentation for emergent behaviour, SLOs for operational outcome.** They answer three different questions and none substitutes for another. The framework's original specification arrived at the same conclusion regarding chaos engineering, and this paper extends it to outcome metrics generally.

---

9. Limitations and Future Work

9.1 A design retraction: the incentive programme

The original specification included a gamified excellence programme — team competition on CRAFT Score, annual titles, public recognition. It has been removed, and the removal is recorded rather than performed silently, because the reason is a finding about the model's deployment envelope: §5.6 identifies that coupling CRAFT Score to team reward converts the index into an optimisation target and invites Goodhart-style gaming at the code level, which agent evaluation relocates rather than eliminates. An instrument author identifying an incentive hazard in his own prior design and withdrawing it is part of the instrument's documentation.

9.2 Unvalidated components

- **MS-1 was not derived by the protocol it ships with** (§3.2.4). It is calibrated expert judgement; the protocol is proposed, not validated. The first full application of the protocol — ideally in a second organisation — is the highest-value next experiment.
- **The degradation base is a convention** (§3.5), with its sensitivity curve owed but its magnitude undefended by data. Coupling-sensitive degradation is deferred.
- **No predictive validation exists.** CRAFT has not been shown to correlate with incident frequency or severity. The framework's claim is that it measures structural conformance, not that conformance predicts outcomes; the correlation study is the obvious and honest next step, and until it exists the paper must not imply it.

9.3 Corpus and evaluator

- The corpus comprises nine services across five public repositories, of which four are reference/teaching codebases and one is the author's own positive-control fixture. No independently produced service landed in the Good or Acceptable tiers; discriminant validity in the middle of the scale is therefore unestablished, and claiming otherwise from this corpus would be overreach.
- No agent-versus-human agreement study exists yet (§5.5); the per-item evidence ledger is the compensating control until it does.
- **The corpus cannot support a meaningful sensitivity analysis.** Its score distribution is bimodal — eight services clustered near zero, one at the ceiling — so weight perturbation produces no tier movement regardless of the model's actual sensitivity (§3.2.5). Obtaining services that score in the Acceptable and Good bands is therefore not merely desirable for discriminant validity; it is a precondition for testing the weights at all. This is the corpus's most consequential limitation and the clearest specification for the next sampling effort.
- **The disconfirming prior stands.** The author's continuous-assurance work found automated gates accepting defective outputs that only human audit caught. The mitigation adopted here (§5.5) bounds but does not eliminate the risk, and the residual is stated rather than argued away.

9.4 Model extensions deferred

- Coupling-sensitive degradation factor
- Journey composition validated against end-to-end failure data
- Profile derivation for a second domain via the §3.2.4 protocol (the §7.2 instantiation)

---

10. Conclusion

CRAFT began as a specification with a documented obstacle: a precise structural index required an evaluator able to read source semantically — flow, autoconfiguration, injection, object interaction — and its author recorded, in 2023, that no such tool existed, recommending expert manual counting instead. Manual counting priced the framework out of use, and it was shelved. The obstacle has since been removed, not by better automation of the old counting but by the arrival of an evaluator that is simultaneously deep enough to read calibration — enabling negative weights, conditional weights, and value bands the model had anticipated but could not compute — and structurally outside the incentive field of the party it evaluates. Nine public-repository audits later, the instrument distinguishes four different ways services fail structurally, reaches its ceiling exactly when a purpose-built fixture earns it, and caught an arithmetic defect in its own profile along the way. The general claim this supports is modest and, we believe, durable: the cost of evidence is a first-class constraint on which governance instruments are buildable at all — and that constraint has just moved.

References

[1] Littlewood, B., & Strigini, L. (1993). Validation of ultra-high dependability for software-based systems. *Communications of the ACM*, 36(11), 69-80.

[2] Musa, J. D., Iannino, A., & Okumoto, K. (1987). *Software Reliability: Measurement, Prediction, Application.* McGraw-Hill.

[3] Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems.* O'Reilly Media.

[4] Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., & Rosenthal, C. (2016). Chaos Engineering. *IEEE Software*, 33(3), 35-41.

[5] RTCA DO-178C (2011). *Software Considerations in Airborne Systems and Equipment Certification.*

[6] ISO 26262 (2018). *Road vehicles — Functional safety.*

[7] IEC 62304 (2006/A1:2015). *Medical device software — Software life cycle processes.*

[8] ISO/IEC 25010 (2011). *Systems and software engineering — SQuaRE — System and software quality models.*

[9] ArchUnit. *A Java architecture test library.* https://www.archunit.org

[10] Carlson, J. L., et al. (2012). *Resilience: Theory and Applications.* Argonne National Laboratory, ANL/DIS-12-1.

Note: standards citations reference the editions the author consulted; readers should verify current editions. Self-citations to the author's prior work on provenance gating, continuous architecture assurance, and multi-agent governance are deliberately given as framework names in prose rather than numbered references in this preprint, pending the DOI cross-references of the artifact release.

Appendix A — Reference Profile MS-1.1

Weights follow the semantic anchor of §3.2.3. Conditional weights use the notation `w [req: prerequisite, else w′]`. Anti-pattern penalties apply wherever the pattern is detected, independent of positive scoring.

A.1 EE — External Entry (max 5)

Weight	Protection
0	None
3	Rate limiter *(mutually exclusive with bulkhead)*
5	Bulkhead *(mutually exclusive with rate limiter)*

A.2 SE — External Exit (max 21)

Weight	Protection
0	None
3	Circuit breaker `[req: timeout, else 1]`
5	Fallback `[req: timeout or circuit breaker, else 2]`
0–2	Retry band: none 0; fixed backoff + cap 1; exponential backoff + jitter + cap 2; +1 global retry budget
4	Timeout
2	Connection pool
4	Idempotent write (idempotency key or naturally idempotent operation)
1	Data compression
1	Asynchronous data dispatch

Penalties: retry without backoff or cap −2; retry over non-idempotent write −2; timeout inversion −3; pool without timeout −1.

A.3 CE — External Consultation (max 15)

Weight	Protection
0	None
3	Circuit breaker `[req: timeout, else 1]`
5	Fallback `[req: timeout or circuit breaker, else 2]`
0–2	Retry band (as A.2)
4	Timeout
2	Connection pool

Penalties: as A.2 where applicable; circular fallback −2; inert circuit breaker −2.

A.4 DI — Internal Data (max 14)

Weight	Protection
0	None
5	Replication
3	Fallback
4	Timeout
2	Connection pool

Penalties: pool without timeout −1.

A.5 AC — Application Container (max 18)

Weight	Protection
0	None (dummy health check)
3	Specific readiness probe
3	Specific liveness probe
3	Self-healing `[req: specific liveness probe, else 0]`
4	Graceful shutdown (SIGTERM handling with connection draining)
2	Declared resource limits (CPU/memory request and limit)
2	Startup probe distinct from liveness
1	Declared disruption policy

Penalties: unconditional liveness (always-200) −3.

Change note vs MS-1: maximum raised from 9 to 18. MS-1's AC vertical saturated at three checkbox probes and contributed up to 21% of a small service's ceiling from its least demanding vertical; graceful shutdown in particular was a genuine omission — absent SIGTERM draining, every deployment discards in-flight requests, a fully structural and fully predictable defect.

A.6 SE-KAFKA — Producer-type asynchronous exit (extension vertical)

Weight	Protection
0	None
3	Schema validation
5	Error handling
1	Throttling
4	Producer idempotence
2	Bounded batch size (institutionally defined)
1	acks=1 *(moderate loss risk — mutually exclusive with acks=all)*
2	acks=all *(minimal loss risk — mutually exclusive with acks=1)*

Change note: acks=all raised from 1 to 2 relative to the original table, which assigned equal weight to both acknowledgement modes despite describing one as strictly safer — an assignment the semantic anchor cannot support.

Appendix B — Worked Calculations

`[Full arithmetic for §6.1, §6.2, and §3.7, step by step. Include the MS-1.1 re-scores of both cases as a second column once the agent corpus run exists.]`

Appendix C — Agent Audit Schema

`[Per scored item: vertical, interaction point identifier, mechanism detected, resolved configuration value, resolution path (file/property/env/default), weight assigned with band or condition applied, penalty flags, evidence trace. This appendix is what makes the method reproducible by others.]`